ЛАБОРАТОРНА РОБОТА №9

Тема: Препроцесори Pug і SCSS. Автоматизація верстки: Vite, PostCSS, оптимізація зображень, контроль якості коду.

Мета: Навчитися будувати професійний робочий процес верстальника: генерувати HTML за допомогою шаблонізатора Pug, організовувати стилі в модульну SCSS-архітектуру, обробляти CSS через PostCSS (Autoprefixer, Browserslist), готувати адаптивні зображення сучасних форматів та автоматично контролювати якість коду за допомогою Prettier, Stylelint і Git hooks.

Вимоги до звіту:

1. Роботу виконувати у `НОВОМУ` репозиторію з назвою `landing-keepsake-vite`.
2. Основа - стартовий шаблон Vite Pug Template.
3. Результат - той самий лендінг Keepsake, що й у лабораторній роботі №7 (завдання 4): макет, дизайн-токени, брейкпоінти, секції, стани й вимоги доступності **залишаються без змін**. Змінюється лише спосіб розробки.
4. Заборонено використання фреймворків та сторонніх UI-бібліотек (Bootstrap, Tailwind тощо).
5. Дотримуватись правил написання html, css та правил іменування файлів.
6. Історія комітів має відображати хід роботи (не один коміт «done» наприкінці).

Самостійна робота:

- Pug: Getting Started · Template Inheritance · Mixins
- Sass: @use · Sass: @forward · Sass Guidelines
- Vite: Getting Started · Vituum: Pug plugin
- PostCSS · Autoprefixer · Browserslist · Vite: CSS
- web.dev: Learn Images · Squoosh · sharp
- Prettier · @prettier/plugin-pug · Stylelint
- husky · lint-staged · commitlint · Conventional Commits
- Vite: Deploying to GitHub Pages

Практична частина

NOTE	Стартовий шаблон містить **лише** Vite + Pug + SCSS. PostCSS-плагіни, оптимізацію зображень, Prettier, Stylelint, husky та lint-staged студент додає й налаштовує **самостійно**. Демонстраційний вміст шаблону (картки «солодощів», зображення мопсів, стилі в `main.scss`) необхідно видалити.

Рекомендована структура проєкту

```
landing-keepsake-vite/
├── .husky/
│   └── pre-commit
├── src/
│   ├── assets/
│   │   ├── images/          # AVIF, WebP, JPEG у кількох ширинах
│   │   └── fonts/           # WOFF2 (якщо шрифти локальні)
│   ├── components/          # header.pug, footer.pug, sections/*.pug
│   ├── data/                # дані секцій (масиви об'єктів)
│   ├── includes/            # head, meta, fonts, svg-спрайт
│   ├── layouts/
│   │   └── main.pug         # базовий макет
│   ├── mixins/              # Pug-міксини
│   ├── pages/
│   │   └── index.pug
│   └── styles/
│       ├── abstracts/       # змінні, функції, міксини (_index.scss з @forward)
│       ├── base/            # reset/normalize, типографіка, шрифти
│       ├── layout/          # контейнер, секції, header, footer
│       ├── components/      # один БЕМ-блок - один файл
│       └── main.scss        # єдина точка входу
├── .browserslistrc          # або поле "browserslist" у package.json
├── .gitignore
├── .prettierrc
├── .prettierignore
├── .stylelintrc.json
├── postcss.config.js
├── vite.config.js
├── package.json
├── package-lock.json
└── README.md
```

Назви папок `utils` / `abstracts` можна залишити як у шаблоні - важливо, щоб структура була логічною й послідовною.

Завдання 1. Підготовка проєкту

1. Створити репозиторій `landing-keepsake-vite` з шаблону (кнопка **Use this template** на GitHub) або клонувати шаблон і підключити власний remote.
2. Перевірити версії `node -v` (LTS, не нижче 20) і `npm -v`, виконати `npm install`, запустити `npm run dev`.
3. Видалити демонстраційний вміст шаблону, залишивши лише структуру.
4. Переконатися, що `.gitignore` містить `node_modules` і `dist`. `package-lock.json` **комітиться**.
5. Розподілити залежності правильно: усе, що потрібне лише для збирання (`vite`, `sass`, `vituum`, плагіни, лінтери), - у `devDependencies`.

TIP У шаблоні `sass` і `@vituum/vite-plugin-pug` знаходяться в `dependencies`. Перенесіть їх у `devDependencies` і поясніть на захисті, чому.

Завдання 2. Pug

Уся розмітка сторінки генерується з Pug-файлів; у `dist/index.html` не має бути жодного рядка, написаного «вручну» в HTML. Мета - **застосувати якомога більше можливостей Pug**, але так, щоб кожна з них мала сенс, а не була «для галочки».

Можливість	Що потрібно зробити
Наслідування шаблонів (`extends`, `block`)	Базовий макет `layouts/main.pug` з блоками щонайменше: `meta`, `title`, `styles`, `content`, `scripts`. Сторінка `pages/index.pug` успадковує макет через `extends` і перевизначає блоки.
`append` / `prepend`	Додати до блоку макета вміст, не замінюючи його (наприклад, `append meta` - `description`, Open Graph-теги; `prepend scripts` або `append styles` - preload шрифту).
`include`	Шапка, футер і кожна секція лендінгу - окремі файли, підключені через `include`. Інлайн-SVG іконки або SVG-спрайт - також через `include`.
Дані	Вміст повторюваних блоків винесено в `src/data/` у вигляді **масивів об'єктів**: пункти навігації, категорії фільтра, роботи портфоліо, послуги, статистика, записи журналу, колонки посилань футера, соцмережі.
Ітерація (`each`)	Усі повторювані елементи виводяться через `each`, зокрема з використанням індексу (`each item, i in items`). Розмітка однієї карточки пишеться **один раз**.
Міксини (`mixin`, `+`)	Щонайменше 4 міксини, винесені в `src/mixins/` і підключені в макеті. Обов'язково: `+btn` (кнопка/посилання з модифікаторами), `+icon(name)`, `+picture(…)` (адаптивне зображення, див. завдання 5), `+work-card(work)`.
Міксин з `block` і `&attributes`	Хоча б один міксин приймає вкладений вміст через `block`, хоча б один - довільні атрибути через `&attributes` (наприклад, `+btn('Get in touch', '#contact')(aria-label="…")`).
Умови (`if` / `else`, `unless`)	Наприклад: активний пункт меню, `loading="lazy"` для всіх зображень, крім першого екрана, необов'язкові поля в даних.
`case` / `when`	Наприклад: вибір SVG-іконки в міксині `+icon` за назвою, або вибір модифікатора кнопки.
Інтерполяція	`#{…}` у тексті, шаблонні рядки `${…}` в атрибутах, умовні класи через об'єкт (`class={'nav__link--active': item.active}`).
Коментарі	Службові коментарі - `//-` (не потрапляють у HTML).

Приклад: дані та вивід карточок робіт

```
//- src/data/works.pug
- const works = [
-   { title: 'Pure Bloom', category: 'branding', image: 'work-01', alt: 'Пакування
```

```
косметики Pure Bloom' },
-   { title: 'Nordic Print', category: 'print', image: 'work-02', alt: 'Серія постерів
Nordic' },
- ]

//- src/components/sections/works.pug
ul.works-grid
  each work, i in works
    li.works-grid__item(data-category=work.category)
      +work-card(work)
```

IMPORTANT

- Фільтр портфоліо на `:checked ~` (з lab-07) зберігається: радіокнопки та чипси також генеруються через `each` з масиву категорій.
- Згенерований HTML має бути валідним (W3C Validator) і семантичним: один `h1`, правильна ієрархія заголовків, `header`/`nav`/`main`/`section`/`footer`.

Завдання 3. SCSS

Вимога	Деталі
Модулі	Лише `@use` і `@forward`. `@import` **заборонено** (він застарів у Dart Sass). Кожна папка має індексний файл (`_index.scss` / `index.scss`), що реекспортує partials через `@forward`. `main.scss` - єдина точка входу, підключається в `layouts/main.pug`.
Partials	Кожен БЕМ-блок - окремий файл `назва-блока.scss`. Файли без на початку в папках модулів не створювати.
Змінні	Дизайн-токени lab-07 (кольори, шрифти, відступи, брейкпоінти) оголошено як SCSS-змінні / map-и в `abstracts/`. Кольори з map-а генеруються в CSS custom properties у `:root` через `@each` (наприклад, `--color-accent`), а в компонентах використовуються через `var(…)`.
Міксин для медіазапитів	Міксин на кшталт `@include mq(md) { … }` з `@content`, що бере значення з map-а брейкпоінтів (`sass:map`). Підхід - **mobile first**: усередині міксина лише `min-width`. Жодних «сирих» `@media (min-width: 768px)` у компонентах.
Функції	Щонайменше одна власна функція (`@function`), наприклад `rem($px)` з використанням `sass:math` (`math.div`).
Інші міксини	Щонайменше 2 корисних міксини, крім медіазапитів (наприклад, `visually-hidden`, `focus-ring`, `flex-center`, `button-reset`). Хоча б один - з параметрами за замовчуванням.
БЕМ	Класи елементів і модифікаторів пишуться через `& і &-- (.card { &title {} &--featured {} }`). Стани - через `&:hover`, `&:focus-visible`, `&.is-open` тощо.
Вкладеність	Не глибше **3 рівнів**. Специфічність - мінімальна, без `id` у селекторах і без `!important`.

Вимога	Деталі
Вендорні префікси	**Не писати вручну** - їх додає Autoprefixer (завдання 4).

Приклад

```
// abstracts/_breakpoints.scss
@use 'sass:map';

$breakpoints: (md: 768px, lg: 1280px);

@mixin mq($bp) {
  @media (min-width: map.get($breakpoints, $bp)) {
    @content;
  }
}

// components/_section-title.scss
@use '../abstracts' as *;

.section-title {
  font-size: rem(18);

  &::after { /* коралова риска */ }

  &--light { color: var(--color-bg-light); }

  @include mq(lg) { letter-spacing: 1.5px; }
}
```

Завдання 4. PostCSS: Autoprefixer і Browserslist

Table 1. Що Vite робить із CSS «з коробки»

Можливість	Як це працює у Vite
Компіляція SCSS	Достатньо встановити пакет `sass` - Vite сам компілює `.scss`, окремий плагін не потрібен.
PostCSS	PostCSS **вбудований** у Vite. Якщо в корені проєкту є `postcss.config.js` (або `.postcssrc`, або поле `postcss` у `package.json`), Vite автоматично застосовує його до всіх CSS/SCSS-файлів - і в `dev`, і в `build`. Без конфігу PostCSS-плагіни не запускаються: **Autoprefixer до складу Vite не входить**.
Мініфікація	Під час `npm run build` CSS мініфікується автоматично (за замовчуванням - через esbuild, параметр `build.cssMinify`). Тому окремий мініфікатор (cssnano) у Vite-проєкті **не потрібен**.

Можливість	Як це працює у Vite
Вендорні префікси	esbuild під час мініфікації додає префікси лише для **кількох** властивостей і орієнтується на `build.cssTarget` (за замовчуванням - сучасні браузери), а **не** на Browserslist. Повноцінні префікси під обраний список браузерів дає лише Autoprefixer.
Інше	Вбудовування `@import`, переписування шляхів у `url()`, хеші в іменах файлів (`style-a1b2c3.css`) для кешування, CSS Modules (`*.module.scss`).

1. Встановити `autoprefixer` у `devDependencies` (сам PostCSS уже є у складі Vite).
2. Створити `postcss.config.js` з Autoprefixer.
3. Описати цільові браузери через Browserslist (поле `"browserslist"` у `package.json` або файл `.browserslistrc`). Мінімум - останні 2 версії Chrome, Firefox, Edge і Safari (з урахуванням iOS Safari). Переглянути фактичний список браузерів: `npx browserslist`. Переглянути, які властивості Autoprefixer префіксуватиме для цього списку: `npx autoprefixer --info`.

```
// postcss.config.js
import autoprefixer from 'autoprefixer';

export default {
  plugins: [autoprefixer()],
};
```

Що показати на захисті

- У `npm run dev` (DevTools → **Styles**) - вендорний префікс, доданий Autoprefixer (наприклад, `-webkit-text-size-adjust` або `-webkit-user-select`, залежно від Browserslist та використаних властивостей).
- Що зміна Browserslist (наприклад, додати `ios_saf >= 12` або прибрати Safari) змінює набір префіксів у результаті.
- Що у `dist/assets/*.css` після `npm run build` CSS мініфікований - і пояснити, хто саме це зробив.

Завдання 5. Зображення: WebP/AVIF, `<picture>`, `srcset`

1. Усі растрові зображення (роботи портфоліо, прев'ю журналу, аватари) підготувати у форматах **AVIF** і **WebP** та запасному **JPEG** (або PNG, якщо потрібна прозорість), кожне - щонайменше у **двох шириних** (наприклад, 1x і 2x, або `400w` і `800w`).
2. Конвертацію виконати одним зі способів:
   - вручну через Squoosh;
   - **(рекомендовано)** npm-скриптом `npm run images` на базі `sharp`, що з папки вихідних зображень генерує всі формати й розміри.
3. Вивід зображень - через Pug-міксин `+picture(…)`, що генерує:
   - `<picture>` з `<source type="image/avif">` і `<source type="image/webp">`, кожен з `srcset` і `sizes`;

- `<img>` з `src` (JPEG), `alt`, `width`, `height` (щоб уникнути зсуву макета), `decoding="async"`;
- `loading="lazy"` для карточок і всього нижче першого екрана; для зображень першого екрана - без `lazy`, можна `fetchpriority="high"`.

4. Іконки й логотип - SVG (інлайн або спрайт), не растр.
5. Переконатися, що після `npm run build` усі зображення є в `dist` і відображаються (шляхи в `srcset` мають обробитися Vite).

Очікуваний HTML після збірки

```
<picture>
  <source type="image/avif" srcset="/assets/work-01-400.avif 400w, /assets/work-01-
800.avif 800w"
          sizes="(min-width: 1280px) 357px, (min-width: 768px) 50vw, 100vw">
  <source type="image/webp" srcset="/assets/work-01-400.webp 400w, /assets/work-01-
800.webp 800w"
          sizes="(min-width: 1280px) 357px, (min-width: 768px) 50vw, 100vw">
  <img class="work-card__img" src="/assets/work-01-800.jpg" alt="Пакування косметики
Pure Bloom"
       width="800" height="632" loading="lazy" decoding="async">
</picture>
```

На захисті показати у DevTools (вкладка **Network**), що браузер завантажує AVIF/WebP потрібного розміру, а зображення карточок підвантажуються лише під час прокручування.

Завдання 6. Якість коду: Prettier, Stylelint, husky, lint-staged

1. **Prettier**: встановити `prettier` і `@prettier/plugin-pug` (без плагіна Prettier не форматує `.pug`). Створити `.prettierrc` (з підключеним плагіном) та `.prettierignore` (`dist`, `node_modules`, `package-lock.json`).
2. **Stylelint**: встановити `stylelint` і `stylelint-config-standard-scss`, створити `.stylelintrc.json`.
3. Налаштувати правила під БЕМ і вимоги роботи: стандартний `selector-class-pattern` дозволяє лише kebab-case і **відхиляє** `__` та `--`, тож його потрібно замінити на БЕМ-шаблон; обмежити вкладеність (`max-nesting-depth`).
4. Додати npm-скрипти:

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview",
  "format": "prettier --write .",
  "lint": "npm run lint:styles && npm run lint:format",
  "lint:styles": "stylelint \"src/**/*.scss\"",
  "lint:format": "prettier --check .",
  "prepare": "husky"
```

```
}
```

5. **husky + lint-staged**: `npm i -D husky lint-staged`, `npx husky init`, у `.husky/pre-commit` - `npx lint-staged`. Конфігурація `lint-staged`: для **`.scss`** - `stylelint --fix` і `prettier --write`; для `.{pug,js,json,md}` - `prettier --write`.
6. **(за бажанням) commitlint**: `@commitlint/cli` + `@commitlint/config-conventional`, хук `.husky/commit-msg`; повідомлення комітів - за Conventional Commits (`feat: add works section`, `fix: header z-index`, `chore: setup stylelint`).
7. **(за бажанням)** `.editorconfig` і рекомендовані розширення редактора у `.vscode/extensions.json`.

Приклад `.stylelintrc.json`

```
{
  "extends": ["stylelint-config-standard-scss"],
  "rules": {
    "selector-class-pattern": [
      "^[a-z][a-z0-9]*(-[a-z0-9]+)*(__[a-z0-9]+(-[a-z0-9]+)*)?(--[a-z0-9]+(-[a-z0-
9]+)*)?$",
      { "resolveNestedSelectors": true, "message": "Клас має відповідати БЕМ:
block__element--modifier" }
    ],
    "max-nesting-depth": 3,
    "selector-max-id": 0,
    "declaration-no-important": true
  }
}
```

На захисті показати, що коміт із помилкою Stylelint (наприклад, клас `.Card_Title` або порожній блок) **блокується** pre-commit хуком.

Завдання 7. Публікація та README

1. Опублікувати зібрану сторінку на GitHub Pages через GitHub Actions за інструкцією Vite: Deploying to GitHub Pages. Для сайту за адресою `https://<user>.github.io/landing-keepsake-vite/` у `vite.config.js` потрібно вказати `base: '/landing-keepsake-vite/'`, інакше стилі та зображення не завантажаться. Папку `dist` у репозиторій **не комітити** - її збирає workflow.
2. Додати `README.md`:
   - посилання на демо (GitHub Pages);
   - вимоги до середовища та команди: `npm install`, `npm run dev`, `npm run build`, `npm run preview`, `npm run format`, `npm run lint`, `npm run images` (якщо є);
   - структуру проєкту й коротке пояснення архітектури Pug і SCSS;
   - перелік використаних інструментів і для чого кожен потрібен;
   - цільові браузери (Browserslist) та порівняння розміру CSS і зображень до/після оптимізації (можна таблицею).

Критерії зарахування роботи

Робота вважається правильно виконаною, якщо на «чистому» клоні репозиторію виконуються всі пункти:

```
git clone <repo> && cd landing-keepsake-vite
npm install        # без помилок
npm run build      # у dist/ - робоча сторінка, ідентична макету lab-07
npm run preview    # сторінка відкривається, стилі та зображення на місці
npm run lint       # 0 помилок Stylelint, Prettier --check без зауважень
```

- ☐ У репозиторії немає `node_modules` і `dist`; є `package-lock.json`.
- ☐ Розмітка повністю генерується з Pug: макет з `block`, `extends`, `append`/`prepend`, `include`, міксини, дані - масиви об'єктів, `each`, умови, `case`.
- ☐ SCSS: `@use`/`@forward` без `@import`, змінні та map-и, міксин медіазапитів, функція, БЕМ через `&__` / `&--`, вкладеність ≤ 3.
- ☐ PostCSS: Autoprefixer + Browserslist; у продакшн-збірці CSS мініфікований (засобами Vite).
- ☐ Зображення: AVIF + WebP + fallback, `<picture>`, `srcset`/`sizes`, `width`/`height`, `loading="lazy"` для карточок.
- ☐ Prettier і Stylelint налаштовані, скрипти `format` і `lint` працюють, pre-commit хук (husky + lint-staged) спрацьовує.
- ☐ Згенерований HTML проходить W3C Validator; виконано всі вимоги lab-07 щодо адаптивності, станів і доступності.
- ☐ Сторінка опублікована на GitHub Pages, є `README.md`.

WARNING	Робота з використанням заборонених фреймворків або згенерована без розуміння коду (студент не може пояснити рішення та конфігурацію інструментів на захисті) не зараховується.

Контрольні запитання

1. Чим відрізняються `dependencies` і `devDependencies`? Чому `sass`, `vite` і лінтери мають бути саме в `devDependencies`?
2. Яку роль відіграє `package-lock.json` і чому його комітять, а `node_modules` - ні? Що робить `npm ci` і чим він відрізняється від `npm install`?
3. Що означають записи `^7.1.7` і `~7.1.7` у `package.json`?
4. Чим препроцесор відрізняється від постпроцесора? Наведіть приклади кожного з вашого проєкту.
5. Чим відрізняються `include` і `extends` у Pug? Як працюють `block`, `append` і `prepend`?
6. Як передати в Pug-міксин вкладений вміст і довільні атрибути? Покажіть на прикладі зі свого проєкту.

7. Чим `//` відрізняється від `//-` у Pug? Як задати клас за умовою?
8. Чому `@import` у Sass застарів? Чим `@use` відрізняється від `@forward` і що таке partials?
9. Як у вашому міксині медіазапитів реалізовано підхід mobile first? Чому значення брейкпоінтів зберігаються в map-і?
10. Чим змінні Sass (`$var`) відрізняються від CSS custom properties (`--var`)? Чому в роботі використовуються обидва види?
11. Чому важливо обмежувати вкладеність у SCSS? Як `&__` і `&--` допомагають дотримуватися БЕМ?
12. Що таке PostCSS і як Vite його підключає? Як Autoprefixer дізнається, які префікси додавати?
13. Що Vite робить із CSS без жодних налаштувань? Чому у Vite-проєкті не потрібен cssnano, але потрібен Autoprefixer?
14. Порівняйте JPEG, PNG, WebP, AVIF і SVG. Чому в `<picture>` джерело AVIF стоїть перед WebP?
15. Як браузер обирає зображення за `srcset` і `sizes`? Навіщо `<img>` атрибути `width` і `height`?
16. Коли використовувати `loading="lazy"`, а коли - `fetchpriority="high"`?
17. Чим Prettier відрізняється від Stylelint? Чому стандартне правило `selector-class-pattern` довелося змінити?
18. Що таке Git hooks? Навіщо поєднувати husky з lint-staged і чому хук не замінює перевірку в CI?
19. Навіщо у `vite.config.js` параметр `base` під час публікації на GitHub Pages?